

Aim: Implement Linked List and demonstrate the functionality to add, delete and search for items in the linked list (Singly, Doubly, Doubly Circular List).

1.Singly Linked List.

CODE:

```
#include <iostream>
using namespace std;
class Node{
    public:
        int data;
        Node *next;
    Node(){
        data = 0;
        next = NULL;
    }
};
class LinkedList{
    Node *head;
    public:
        LinkedList(){
            head = NULL;
        }
        void insertNode(int val){
            Node *t = new Node();
            t->data = val;
            t->next = NULL;
            if(head == NULL){
                head = t;
                return;
            }
            Node *n = head;
            while(n->next != NULL){
                n = n->next;
            }
            n->next = t;
        }
        void searchNode(int val){
            Node *t= head;
            if(head == NULL){
                cout<<"Linked List doesn't exist!";
                return;
            }
            while(t != NULL){
                if(t->data == val){
                    cout<<val<<" is present!"<<endl;
                    return;
                }
                t = t->next;
            }
        }
    }
```

```
        cout<<"Value doesn't exist!"<<endl;
    }
    void display(){
        Node *t = head;
        if(head == NULL){
            cout<<"Linked list doesn't exist!"<<endl;
            return;
        }
        cout<<"Linked List: ";
        while(t != NULL){
            cout<<t->data<<" ";
            t = t->next;
        }
        cout<<endl;
    }
    void deleteNode(int val){
        Node *t = head;
        Node *p = head;
        if(head == NULL){
            cout<<"Linked List don't exist!"<<endl;
            return;
        }
        if(t->data == val){
            head = head->next;
            delete t;
            return;
        }
        t = t->next;
        while(t->next!= NULL){
            if(t->data == val){
                p->next = t->next;
                delete t;
                return;
            }
            p = t;
            t = t->next;
        }
        if(t->data == val){
            p->next = t->next;
            delete t;
        }
        else{
            cout<<"Value doesn't exist in Linked List"<<endl;;
        }
    }
};
int main(){
    int n,ch;
```

```
LinkedList list;
while(true){
    cout<<"Enter the number for the Linked List operation->"<<endl;
    cout<<"1.Value enter:"<<endl;
    cout<<"2.Display:"<<endl;
    cout<<"3.Delete a value:"<<endl;
    cout<<"4.Search value:"<<endl;
    cout<<"5.Exit:"<<endl;
    cout<<"Enter the choice:";
    cin>>ch;
    switch(ch){
        case 1:
            cout<<"Enter the value for the Linked List:";
            cin>>n;
            list.insertNode(n);
            break;
        case 2:
            list.display();
            break;
        case 3:
            cout<<"Enter the value to delete:";
            cin>>n;
            list.deleteNode(n);
            break;
        case 4:
            cout<<"Enter the value:";
            cin>>n;
            list.searchNode(n);
            break;
        case 5:
            cout<<"Exiting from the program...";
            return 0;
            break;
        default:
            cout<<"Invalid operation for Linked List!!";
    }
}
return 0;
}
```

OUTPUT:

```

C:\Users\ROHIT\OneDrive\De  ×  +  v
Enter the number for the Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:55
Enter the number for the Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:56
Enter the number for the Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:57
Enter the number for the Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:2
Linked List: 55 56 57

Enter the number for the Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:3
Enter the value to delete:56
Enter the number for the Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:4
Enter the value:56
Value doesn't exist!
Enter the number for the Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:5
Exiting from the program...
-----
Process exited after 43.17 seconds with return value 0
Press any key to continue . . . |

```

2.Singly Linked Circular List.**CODE:**

```

#include<iostream>
using namespace std;
class Node{
    public:
        Node *next;
        int data;
        Node(){
            next = NULL;
            data = 0;
        }
};
class SLinkedListC{
    Node *head;
    public:
        SLinkedListC(){
            head = NULL;
        }
        void insertNode(int val){
            Node *t = new Node();
            t->data = val;
            if(head == NULL){
                head = t;
                t->next = t;
            }
        }
};

```

```
        return;
    }
    Node *p = head;
    while(p->next != head){
        p = p->next;
    }
    p->next = t;
    t->next = head;
}

void display(){
    Node *t = head;
    if(head == NULL){
        cout<<"Linked List doesn't exist"<<endl;
        return;
    }
    cout<<"Singly Linked Circular List:";
    do{
        cout<<t->data<<" ";
        t = t->next;
    }while(t!=head);
    cout<<endl;
}

void deleteNode(int val){
    Node *t = head;
    Node *p = head;
    if(head == NULL){
        cout<<"Linked List doesn't exist"<<endl;
        return;
    }
    if(val == t->data){
        while(p->next != head){
            p = p->next;
        }
        p->next = head->next;
        Node *temp = head;
        head = head->next;
        delete temp;
        return;
    }
    p = t;
    t = t->next;
    while(t != head){
        if(t->data == val){
            p->next = t->next;
            delete t;
            return;
        }
        p = t;
        t = t->next;
    }
}
```

```
t = t->next;
}
cout << "Value not found in the Singly Linked Circular List!"<<endl;
}
void searchNode(int val){
    Node *t= head;
    if(head == NULL){
        cout<<"Linked List doesn't exist!"<<endl;
        return;
    }
    while(t != head){
        if(t->data == val){
            cout<<val<<" is present!"<<endl;
            return;
        }
        t = t->next;
    }
    cout<<"Value doesn't exist!"<<endl;
}

};
int main(){
    int n,ch;
    SLinkedListC list;
    while(true){
        cout<<"Enter the number for the Singly Linked Circular List operation->"<<endl;
        cout<<"1.Value enter:"<<endl;
        cout<<"2.Display:"<<endl;
        cout<<"3.Delete a value:"<<endl;
        cout<<"4.Search value:"<<endl;
        cout<<"5.Exit:"<<endl;
        cout<<"Enter the choice:";
        cin>>ch;
        switch(ch){
            case 1:
                cout<<"Enter the value for the Linked List:";
                cin>>n;
                list.insertNode(n);
                break;
            case 2:
                list.display();
                break;
            case 3:
                cout<<"Enter the value to delete:";
                cin>>n;
                list.deleteNode(n);
                break;
            case 4:
                cout<<"Enter the value:";
```

```

        cin>>n;
        list.searchNode(n);
        break;
    case 5:
        cout<<"Exiting from the program...";
        return 0;
        break;
    default:
        cout<<"Invalid operation for Linked List!!";
    }
}
return 0;
}

```

OUTPUT:

```

Enter the number for the Singly Linked Circular List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:55
Enter the number for the Singly Linked Circular List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:56
Enter the number for the Singly Linked Circular List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:57
Enter the number for the Singly Linked Circular List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:2
Singly Linked Circular List:55 56 57
Enter the number for the Singly Linked Circular List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:5
Exiting from the program...
-----
Process exited after 28.58 seconds with return value 0
Press any key to continue . . . |

```

3.Doubly Linked List.**CODE:**

```

#include <iostream>
using namespace std;
class Node{
public:
    int data;
    Node *next, *prev;
    Node(){
        data = 0;
        next = NULL;
        prev = NULL;
    }
};
class DoublyLinkedList{
    Node* head;

```

```
public:
    DoublyLinkedList(){
        head = NULL;
    }
    void display(){
        Node *t=head;
        if(head == NULL){
            cout<<"Doubly Linked List doesn't exist"<<endl;
            return;
        }
        cout<<"Doubly Linked List:";
        while(t != NULL){
            cout<<t->data<<" ";
            t = t->next;
        }
        cout<<endl;
    }
    void searchNode(int val){
        Node *t= head;
        if(head == NULL){
            cout<<"Doubly Linked List doesn't exist!"<<endl;
            return;
        }
        while(t != NULL){
            if(t->data == val){
                cout<<val<<" is present!"<<endl;
                return;
            }
            t = t->next;
        }
        cout<<"Value doesn't exist!"<<endl;
    }
    void insertNode(int val){
        Node *t = new Node();
        t->data = val;
        if(head == NULL){
            head = t;
            return;
        }
        Node *n = head;
        while(n->next != NULL){
            n = n->next;
        }
        n->next = t;
        t->prev = n;
    }
    void deleteNode(int val){
        if(head == NULL){
```



```

        cout<<"Doubly Linked List doesn't exist"<<endl;
        return;
    }
    Node *t = head,*p = head;
    if(t->next == NULL){
        if(t->data == val){
            delete(t);
            head = NULL;
            return;
        }
        else{
            cout<<"Value not found"<<endl;
            return;
        }
    }
    else{
        if(t->data ==val){
            head = head->next;
            delete t;
            head->prev = NULL;
            return;
        }
        else{
            t = t->next;
            while(t->next != NULL){
                if(t->data == val){
                    p->next = t->next;
                    t->next->prev = p;
                    delete t;
                    return;
                }
                p = t;
                t = t->next;
            }
        }
    }
    if(t->data == val){
        p->next = NULL;
        delete t;
        return;
    }
    else{
        cout<<"Value not found"<<endl;
    }
}

};
int main(){
    int n,ch;

```

```
DoublyLinkedList list;
while(true){
    cout<<"Enter the number for the Doubly Linked List operation->"<<endl;
    cout<<"1.Value enter:"<<endl;
    cout<<"2.Display:"<<endl;
    cout<<"3.Delete a value:"<<endl;
    cout<<"4.Search value:"<<endl;
    cout<<"5.Exit:"<<endl;
    cout<<"Enter the choice:";
    cin>>ch;
    switch(ch){
        case 1:
            cout<<"Enter the value for the Linked List:";
            cin>>n;
            list.insertNode(n);
            break;
        case 2:
            list.display();
            break;
        case 3:
            cout<<"Enter the value to delete:";
            cin>>n;
            list.deleteNode(n);
            break;
        case 4:
            cout<<"Enter the value:";
            cin>>n;
            list.searchNode(n);
            break;
        case 5:
            cout<<"Exiting from the program...";
            return 0;
            break;
        default:
            cout<<"Invalid operation for Linked List!!";
    }
}
return 0;
}
```

OUTPUT:

```

C:\Users\ROHIT\OneDrive\De x + v
Enter the number for the Doubly Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:55
Enter the number for the Doubly Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:56
Enter the number for the Doubly Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:57
Enter the number for the Doubly Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:2
Doubly Linked List:55 56 57

Enter the number for the Doubly Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:3
Enter the value to delete:56
Enter the number for the Doubly Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:4
Enter the value:56
Value doesn't exist!
Enter the number for the Doubly Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:4
Enter the value:55
55 is present!
Enter the number for the Doubly Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:2
Doubly Linked List:55 57
Enter the number for the Doubly Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:5
Exiting from the program...
-----
Process exited after 37.1 seconds with return value 0
Press any key to continue . . . |

```

4.Doubly Linked Circular List.

CODE:

```

#include <iostream>
using namespace std;

```

```

class Node {
public:
    int data;
    Node *next, *prev;
    Node() {
        data = 0;
        next = NULL;
        prev = NULL;
    }
};

```

```

class DoublyLinkedList {
    Node* head;
public:
    DoublyLinkedList() {
        head = NULL;
    }
}

```

```
void display() {
    if (head == NULL) {
        cout << "Doubly Circular Linked List doesn't exist" << endl;
        return;
    }
    Node *t = head;
    cout << "Doubly Circular Linked List: ";
    do {
        cout << t->data << " ";
        t = t->next;
    } while (t != head);
    cout << endl;
}
```

```
void searchNode(int val) {
    if (head == NULL) {
        cout << "Doubly Circular Linked List doesn't exist!" << endl;
        return;
    }
    Node *t = head;
    do {
        if (t->data == val) {
            cout << val << " is present!" << endl;
            return;
        }
        t = t->next;
    } while (t != head);
    cout << "Value doesn't exist!" << endl;
}
```

```
void insertNode(int val) {
    Node *t = new Node();
    t->data = val;
    if (head == NULL) {
        head = t;
        head->next = head;
        head->prev = head;
        return;
    }
    Node *last = head->prev;
    last->next = t;
    t->prev = last;
    t->next = head;
    head->prev = t;
}
```

```
void deleteNode(int val) {
    if (head == NULL) {
```

```
        cout << "Doubly Circular Linked List doesn't exist" << endl;
        return;
    }
    Node *t = head;

    // Only one node
    if (head->next == head) {
        if (head->data == val) {
            delete head;
            head = NULL;
            return;
        } else {
            cout << "Value not found" << endl;
            return;
        }
    }

    // If head node is to be deleted
    if (head->data == val) {
        Node *last = head->prev;
        Node *nextNode = head->next;
        last->next = nextNode;
        nextNode->prev = last;
        delete head;
        head = nextNode;
        return;
    }

    // Traverse to find the node
    Node *p = NULL;
    do {
        if (t->data == val) {
            p->next = t->next;
            t->next->prev = p;
            delete t;
            return;
        }
        p = t;
        t = t->next;
    } while (t != head);

    cout << "Value not found" << endl;
}

};

int main() {
    int n, ch;
    DoublyLinkedList list;
```

```
while (true) {
    cout << "Enter the number for the Doubly Circular Linked List operation->" << endl;
    cout << "1.Value enter:" << endl;
    cout << "2.Display:" << endl;
    cout << "3.Delete a value:" << endl;
    cout << "4.Search value:" << endl;
    cout << "5.Exit:" << endl;
    cout << "Enter the choice:";
    cin >> ch;
    switch (ch) {
    case 1:
        cout << "Enter the value for the Linked List:";
        cin >> n;
        list.insertNode(n);
        break;
    case 2:
        list.display();
        break;
    case 3:
        cout << "Enter the value to delete:";
        cin >> n;
        list.deleteNode(n);
        break;
    case 4:
        cout << "Enter the value:";
        cin >> n;
        list.searchNode(n);
        break;
    case 5:
        cout << "Exiting from the program...";
        return 0;
    default:
        cout << "Invalid operation for Linked List!!";
    }
}
return 0;
}
```

OUTPUT:

```
C:\Users\ROHIT\OneDrive\De  X  +  v
Enter the number for the Doubly Circular Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:55
Enter the number for the Doubly Circular Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:56
Enter the number for the Doubly Circular Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:57
Enter the number for the Doubly Circular Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:1
Enter the value for the Linked List:58
Enter the number for the Doubly Circular Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:2
Doubly Circular Linked List: 55 56 57 58
Enter the number for the Doubly Circular Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:3

Enter the value to delete:58
Enter the number for the Doubly Circular Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:4
Enter the value:58
Value doesn't exist!
Enter the number for the Doubly Circular Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:2
Doubly Circular Linked List: 55 56 57
Enter the number for the Doubly Circular Linked List operation->
1.Value enter:
2.Display:
3.Delete a value:
4.Search value:
5.Exit:
Enter the choice:5
Exiting from the program...
-----
Process exited after 116.1 seconds with return value 0
Press any key to continue . . . |
```